Parallel OCR Pipeline

Parallel Programming Project
CSC611M G01

Adrian Denzel M. Macayan,
Luis Andrew S. Madridijo

Project Overview

OCR on many images is slow when sequential, because each image is loaded, preprocessed and run through the model in turn.

Idea: A parallel pipeline lets several images be processed at once.

Solution: loader → image queue → n workers (preprocess + OCR) → CSV.

Tech Stack: Python 3.14 with multiprocessing (stdlib only); Pillow for preprocessing; Tesseract 5.5.3 via pytesseract. rapidfuzz to score accuracy

Main Results:

- All 100 images are processed, with 99.28% character recall
- Best-practical speedup is 5.40x at 10 workers

Data and Data Preprocessing + Processing

Dataset:

- 100 PNG word images, each with a ground-truth label in labels.txt
- 838 characters in total. Words run 3–15 letters.
- Images are 100 px tall and 200–397 px wide. 99 are RGB and 1 is RGBA

Loading and Input Handling:

- A single loader process reads the .png files from image_dir in sorted order.
- Each image is fully loaded into memory (img.load()) before its file closes,
- The loader puts each (filename, image) pair on the shared queue for the workers.
- When it finishes, it queues one **None** sentinel per worker, which signals each worker to stop.

Preprocessing Steps:

- Transparent pixels are flattened onto a white background (RGBA → RGB).
- The pipeline then runs grayscale → autocontrast → 3×3 median filter (denoise).

Inference, Post-processing and Output

Inference (Tesseract OCR)

- Each worker calls pytesseract.image_to_string(img, lang="eng") on its preprocessed image.
- The English model comes from the project's local eng.traineddata.
- Every worker launches its own Tesseract process, limited to one thread.

Post-Processing

- Leading and trailing whitespace and newlines are stripped.
- A regex keeping letters only since the dataset labels are alphabetic.
- Letter case is preserved.
- The cleaned text is what gets scored against labels.txt.

Output

- One output CSV holding all results, with columns ID, filename, extracted text and processing time.
- Processing time is the OCR time per image in milliseconds.
- Each worker appends its row under a lock, and the ID is assigned inside the same locked block. The result is 100 rows with unique sequential IDs (0–99).
- Rows are written as images finish, so they are not in filename order.

Inference Results

```
Base Pipeline Test
Time for 8 workers: 1.7748 seconds
Character recall: 832/838 = 99.28%
Exact word matches: 92/100
```

Parallelization Methodology — Decomposition

- **Data Parallelism:** Each image is an independent task with no dependencies, so different images can be processed at the same time.
- **Pipeline Overlap:** Loading runs alongside processing, so the loader keeps loading while workers are already doing OCR.
- **One-Worker/Image Workflow**: Preprocessing and inference are not split, and the same worker does both for an image
- **Serial Sections**
  - Singe Loader
  - Shared CSV Write

Parallelization Methodology — Process Approach

- **Workers are Processes**
  - Preprocessing and OCR are CPU-bound,
  - Separate interpreters avoid GIL contention in the Python-side image work.
  - Each process starts as a fresh interpreter and re-imports the modules (Windows OS)
- Each worker launches its own tesseract.exe
- Images are copied between processes through the queue rather than shared in memory.

Parallelization Methodology — Worker Model

- **One Loader (Producer)**
- **n OCR Workers (Consumer)**
  - Runs the same work function
  - Every process only has one thread including running Tesseract.
- **Work Distribution**
  - Dynamic Scheduling via Shared Queue i.e. whichever worker is idle takes the next image.
  - No worker waits on another. CSV rows appear in completion order.
  - This balances uneven work. Image widths vary from 200 to 397 px, and OCR time varies from about 71 to 104 ms per image.

Synchronization Methodology — Shared Resources

- **Image_queue:** a multiprocessing.Queue carrying images from the loader to the workers.
- **output_lock**: a Lock that protects the output file and the ID counter.
- **id_counter**: a shared Value("i") used to hand out sequential row IDs.
- **ocr_results.csv**: the file every worker appends to. All of these are created in the parent and passed to each process.

Synchronization Methodology — Producer-Consumer

- **Producer**
  - Loader puts (filename, image) pairs on the queue.
- **Consumer**
  - Workers take them with image_queue.get().
  - get() blocks when the queue is empty, so idle workers sleep and don’t poll. They wake as soon as an item arrives.
- **Queue Implementation**
  - It has its own internal locking, so put and get need no extra lock.
  - No size limit, so the loader can run ahead of the work.

Synchronization Methodology — Mutual Exclusion

- **Critical Sections**
  - Reading the ID
  - Incrementing ID
  - Writing the CSV Row
- **with** releases the lock automatically
- OCR runs outside the lock, so only the short write is serialized

Synchronization Methodology — Clean Termination

- After the last image, the loader puts one None sentinel per worker.
  - A worker stops when it receives one.
  - One sentinel per worker means each worker gets exactly one stop signal.
- The parent joins the loader first, then every worker.
  - This cannot deadlock because the workers keep draining the queue while the loader finishes.

Benchmarking Methodology — Metrics

- **Tn**: mean wall-clock time (ms) with n workers.
  - Incl. Loading, Process startup, OCR, CSV writing and Shutdown.
  - Speedup: S = T1 / Tn, where T1 is the same pipeline with 1 worker.
- **Efficiency**: E = S / n, where 100% is perfect scaling.
- **Accuracy check:** character recall on every run.

The optimal configuration is the fewest workers within 5% of the best mean time.

Benchmarking Methodology — Experimental Design

- Worker counts are 1, 2, 4, 6, 8, 10, 12, 16, 20 and 24, which goes past the 20 logical cores to show and simulate oversubscription.
- Each count runs 30 times and the mean is reported.
- Runs are interleaved (1, 2, 4, … 24, then repeat), to reduce any noise cause by drift in system load.
- One untimed warm-up run comes to removed initial cold file cache slowdown.

Benchmarking Experimental Environment

CPU: **Intel Core Ultra 265KF**

- 8 Performance cores up to 5.5 GHz
- 12 Efficient cores up to 4.6 GHz
- 20 Total Threads
- 30 MB Intel® Smart Cache
- 36 MB L2 Cache

OS: Windows 11

Software: Python 3.14.3 (w/ Pillow, Pytesseract, Rapidfuzz) , Tesseract 5.5.3.

Only Non-Background User Process and App Running is Python

Benchmarking Results (mean of 30 runs per worker count)

Workers	Mean Tn (ms)	Speedup	Efficiency	Recall %
1	7826.3	1.00	100%	99.28
2	4137.6	1.89	95%	99.28
4	2368.3	3.30	83%	99.28
6	1802.7	4.34	72%	99.28
8	1556.5	5.03	63%	99.28
10	1449.3	5.40	54%	99.28
12	1422.0	5.50	46%	99.28
16	1489.2	5.26	33%	99.28
20	1697.9	4.61	23%	99.28
24	1852.3	4.23	18%	99.28

Parallel Programming Insights

Results:

- S = T1 / Tn, with T1 = 7826.3 ms (1 worker)
- Fastest: 12 workers (S = 5.50x)
- Optimal: 10 workers (S = 5.40x, E = 54%)

Conclusions

- Parallelization gives a real but sublinear gain.
- Fixed startup cost, not CPU capacity, caps the speedup.
  - Theoretically, a Linux implementation would be faster as process creation is cheaper there.
- Future solution with multiple producers i.e. loader?

Parallel OCR Pipeline

Parallel Programming Project
CSC611M G01

Adrian Denzel M. Macayan,
Luis Andrew S. Madridijo